

DATA ENGINEERING

Assignment 3: Advanced SQL Querying

Joins • Set Operations • Subqueries • CTEs

Assignment Goal

Combine multiple datasets using advanced JOIN operations, compare datasets using set operations, and construct complex multi-step transformations using Subqueries and Common Table Expressions (CTEs).

Learning Outcomes

- Utilize INNER, LEFT, RIGHT, and FULL OUTER joins appropriately.
- Apply set operations (UNION, EXCEPT, INTERSECT) to compare query results.
- Write correlated and non-correlated subqueries in SELECT, FROM, and WHERE clauses.
- Break down complex business logic into readable, maintainable CTEs.

Database Setup Instructions (MANDATORY)

Ensure you have successfully executed the 'Northwind' installation script as instructed in Assignment 2.

- Download Link:

<https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs>

Begin your assignment script with: USE Northwind; GO

General Instructions

- Write your queries using standard SQL Server syntax.
- Read the business rules carefully to deduce the table relationships before writing queries.
- Submit a single .sql script containing all solutions.

Task 1 - Joins & Set Operations

Weight: **40 marks**

1.1 Write a query using a LEFT JOIN combined with an IS NULL condition (Anti-Join pattern) between the [Customers] and [Orders] tables to retrieve the 'CustomerID' and 'CompanyName' of all customers who have NEVER placed an order.

1.2 Write a query using a Set Operation (UNION) to return a single column named 'CityList'. This list must contain all unique cities where either a customer is located ('City' from [Customers]) OR an employee is located ('City' from [Employees]).

1.3 Write a query using a SELF JOIN on the [Employees] table. The query must display each employee's 'FirstName' and 'LastName' concatenated and aliased as 'EmployeeName', positioned directly next to their manager's 'FirstName' and 'LastName' concatenated and aliased as 'ManagerName'. (Hint: Use the 'ReportsTo' column).

Task 2 - Subqueries

Weight: **30 marks**

2.1 Write a query using a non-correlated subquery in the WHERE clause to retrieve the 'ProductID', 'ProductName', and 'UnitPrice' of all products whose price is strictly greater than the overall average 'UnitPrice' of ALL products in the [Products] table.

2.2 Write a query using a correlated subquery in the SELECT clause to retrieve the 'CustomerID' and 'CompanyName' from the [Customers] table, and directly alongside it, a calculated column named 'LatestOrderDate' which fetches the maximum (most recent) 'OrderDate' for that specific customer from the [Orders] table.

Task 3 - Common Table Expressions (CTEs)

Weight: **30 marks**

3.1 Create a CTE named 'CategoryAverages' that computes the average 'UnitPrice' per 'CategoryID' from the [Products] table. Immediately following the CTE, write a main query that INNER JOINS this CTE with the [Products] and [Categories] tables to display the 'ProductName', 'CategoryName', the product's actual 'UnitPrice', and the CTE's calculated average aliased as 'AvgCategoryPrice'.

3.2 Extend the exact logic from Task 3.1: In your main SELECT statement, add a calculated column named 'PriceDifference' that subtracts the 'AvgCategoryPrice' (retrieved from the CTE) from the product's actual 'UnitPrice'.

Marking Rubric

Area	Marks	Evaluation focus
Task 1: Joins & Sets	40	Correct join types, proper handling of NULLs in anti-joins, accurate set operations.
Task 2: Subqueries	30	Correct execution context (correlated vs non-correlated), accurate logical filtering.
Task 3: CTEs	30	Readable WITH syntax, proper scope usage, accurate mathematical calculations.
Total	100	Efficient logic and clean multi-step query structures conforming to the standard database.

Submission Checklist

- Use the Northwind database installed in Assignment 2.
- Begin the script with `USE Northwind; GO`.
- Use the specified JOIN, set-operation, subquery, and CTE patterns.
- Submit one .sql script containing all solutions.